

UM PASSEIO INICIAL E ANALÍTICO NOS ALGORITMOS DE ORDENAÇÃO: INSERTION SORT, MERGE SORT, SELECTION SORT

Pedro Cairo Bernardo de Oliveira

Trabalho apresentado à disciplina de Algoritmos

RESUMO

O relatório apresenta uma análise comparando o desempenho de três algoritmos de ordenação (Insertion Sort, Selection Sort e o Merge Sort), todos implementados na linguagem C++. A meta dessa atividade é avaliar o tempo de execução de cada algoritmo em função do tamanho da entrada e da forma como as entradas foram definidas, seja com dados em ordem aleatória, já ordenados, de trás pra frente, quase totalmente ordenados e dados quase ordenados na ordem inversa. A metodologia consistiu em implementar a lógica dos 3 algoritmos, executar os testes a partir dos resultados obtidos, gerar o gráfico comparativo em cada caso. No fim, os resultados bateram com o que foi estudado teoricamente durante esse semestre. O Merge Sort mantém ótimo desempenho e estabilidade para todos os 5 casos, com sua complexidade sendo $O(n \log n)$. O Selection Sort mostrou complexidade $O(n^2)$ em todos os casos, independentemente de como vinham ordenados os dados, e por fim, o Insertion Sort, que também tem complexidade $O(n^2)$, se mostrou melhor em casos onde os dados vinham quase todos ordenados ou quase ordenados, chegando próximo a $O(n)$, e em contramão, apresentou seu pior desempenho quando os dados estão de trás pra frente.

1. Introdução

Esse relatório tem a função de comparar o desempenho entre três algoritmos, o Insertion Sort, e Selection Sort e o Merge Sort. A ideia é mostrar a eficiência e a utilização correta de cada um deles para determinado tipo de tarefa, pondo em prática tudo o que vimos nesse semestre na disciplina do nosso querido professor.

2. Metodologia

Os três algoritmos foram implementados em linguagem C++. Um programa de testes (teste_ordenacao.cpp) foi utilizado para ler um arquivo de entrada, aplicar o método de ordenação escolhido (seleção, inserção ou merge sort) e medir o tempo de execução em microssegundos.

2.1 Implementação dos algoritmos

A seguir são mostrados os trechos de código de cada algoritmo implementado, que serão referenciados ao longo da análise dos resultados.

A função ordenado (Código 1) percorre o vetor uma única vez comparando cada elemento com o anterior (linha 3); caso encontre algum par fora de ordem, retorna false imediatamente. Ela foi usada para validar e garantir que cada algoritmo realmente estaria cumprindo os seus papéis ao fim da execução dos testes.

```
bool ordenado(int a[], unsigned int t) {
    for (unsigned int i = 1; i < t; i++) {
        if (a[i]<a[i-1]) {
            return false;
        }
    }
    return true;
}
```

Código 1 — Função ordenado(), usada para validar a saída dos algoritmos.

A função selecao (Código 2) implementa a Selection Sort de forma direta: O laço externo começa no índice 0 e vai até o penúltimo elemento, definindo a posição inicial onde o menor elemento daquela rodada deve ser colocado. O código divide o array em ordenados, à esquerda e desordenados para a direita. O código admite que o elemental atual é o menor de todos e o guarda em min. Já o laço interno inicia uma posição à frente do externo, seguindo até o final comparando dois valores em busca de achar o menor índice para atribuir a variável min. Depois de terminado, o algoritmo faz uma troca por rodada, substituindo o valor de a[i] por a[min].

```
void selecao(int a[], unsigned int t) {
```

```

for (unsigned int i = 0; i < t - 1; i++) {
    unsigned int min = i;
    for (unsigned int j = i+1; j < t; j++) {
        if (a[j] < a[min]) {
            min = j;
        }
    }
    int aux = a[i];
    a[i] = a[min];
    a[min] = aux;
}

```

Código 2 — Função *selecao()*, implementação da Ordenação por Seleção.

A função *insercao* (Código 3), por outro lado, possui uma diferença estrutural importante em relação à seleção: nós já definimos o laço externo começando do índice 1, considerando que um único elemento por si só, já está ordenado. O laço interno (Linha 4), é iniciado a cada início de rodada e ele começa uma posição atrás do laço externo e vai caminhando para trás, em direção ao início do vetor. Criamos a variável “idx” (Linha 3) a fim de acompanhar a posição do elemento que está sendo movido e sempre que o elemento atual é menor que o seu vizinho da esquerda, eles trocam de lugar. Após a troca, o idx decrementa para que assim ele continue no índice do valor correto. O break tem o papel de poupar verificações desnecessárias, caso encontre um que seja menor ou igual ao valor que está sendo movido.

```

void insercao(int a[], unsigned int t) {
    for (unsigned int i=1; i < t; i++) {
        int idx = i;
        for (int j = i-1; j > -1; j--){
            if (a[idx] < a[j]) {
                int aux = a[idx];
                a[idx] = a[j];
                a[j] = aux;
                idx--;
            }
            else {
                break;
            }
        }
    }
}

```

Código 3 — Função *insercao()*, implementação da Ordenação por Inserção.

Por fim, o Merge Sort (Código 4) baseia-se na estratégia de “Divisão e Conquista”, que separa o vetor em partes menores, recursivamente, até que sejam itens únicos para depois ir juntando par a par de forma ordenada. O código calcula o ponto médio do vetor ($(\text{meio} = \text{inicio} + \text{fim}) / 2$), e a partir disso faz chamadas recursivas para dividir repetidamente até que restem apenas subvetores de 1 elemento. Após isso, o código começa a intercalar, criando um vetor temporário (temp) para armazenar os elementos já ordenados. O primeiro laço while compara os elementos da metade

esquerda com os da metade direita a fim de copiar o menor valor entre eles, avançando os índices caso copiado. Os próximos dois while esvaziam qualquer elemento que tenha sobrado, e por fim o último copia todos os elementos organizados de temp para o vetor original e libera a memória com (delete[] temp).

```
void merge(int a[], int i1, int j1, int i2, int j2) {
    int *temp = new int[((j1 - i1) + (j2 - i2) + 2)];
    int i, j, k;
    i = i1;
    j = i2;
    k = 0;

    while (i <= j1 and j <= j2) {
        if (a[i] < a[j]) {
            temp[k++] = a[i++];
        }
        else {
            temp[k++] = a[j++];
        }
    }

    while (i <= j1) {
        temp[k++] = a[i++];
    }

    while (j <= j2) {
        temp[k++] = a[j++];
    }

    for (i = i1, j = 0; i <= j2; i++, j++) {
        a[i] = temp[j];
    }

    delete[] temp;
}

void merge_sort_r(int a[], int inicio, int fim) {
    if (inicio < fim) {
        int meio = (inicio + fim) / 2;
        merge_sort_r(a, inicio, meio);
        merge_sort_r(a, meio + 1, fim);
        merge(a, inicio, meio, meio + 1, fim);
    }
}

void merge_sort(int a[], unsigned int t) {
    if (t > 1) {
        merge_sort_r(a, 0, t - 1);
    }
};
```

Código 4 — Funções merge(), merge_sort_r() e merge_sort(), implementação do Merge Sort.

Foram gerados cinco conjuntos de arquivos de teste (Caso 01 a Caso 05), cada um contendo dez arquivos com tamanhos de entrada variando de 10.000 a 100.000 elementos, em incrementos de 10.000. Os cinco casos representam padrões de entrada diferentes:

Caso 01: dados dispostos de forma aleatória, sem nenhuma ordenação prévia.

Caso 02: dados já ordenados de forma crescente (melhor caso teórico para algoritmos adaptativos).

Caso 03: dados ordenados de forma decrescente, ou seja, em ordem inversa à desejada (pior caso teórico para a Ordenação por Inserção).

Caso 04: dados organizados em blocos crescentes, com pequenas desordens locais dentro de cada bloco (cenário "quase ordenado").

Caso 05: dados organizados em blocos decrescentes, com pequenas desordens locais (cenário "quase ordenado", porém na direção inversa).

Cada algoritmo foi executado sobre os dez tamanhos de entrada de cada um dos cinco casos, totalizando 150 execuções (3 algoritmos $\times$ 5 casos $\times$ 10 tamanhos). Os tempos coletados foram convertidos de microssegundos para milissegundos e organizados em planilhas eletrônicas, a partir das quais foram gerados gráficos de linha, com o tamanho da entrada no eixo X e o tempo de ordenação (em milissegundos) no eixo Y.

3. Desenvolvimento

Nesta seção são apresentados os dados coletados e os gráficos gerados para cada um dos cinco casos de teste, relacionando o comportamento observado com a estrutura de código apresentada na Seção 2.1.

3.1 Caso 01 — Dados Aleatórios

No cenário de dados aleatórios, os algoritmos de Insertion Sort (Código 3) e Selection Sort (Código 2) apresentam desempenho parecidos entre si, os dois crescem em ($O(n^2)$) com o aumento do tamanho da entrada. Como os dados são embaralhados, a condição de parada antecipada da inserção (break na linha 12 do Código 3) dificilmente é utilizada, fazendo com que ela se comporte de forma parecida com o Selection Sort, desperdiçando seu potencial nesses casos. O Merge Sort (Código 4), por sua vez, se mantém constante na escala do gráfico, confirmando sua complexidade $O(n \log n)$.

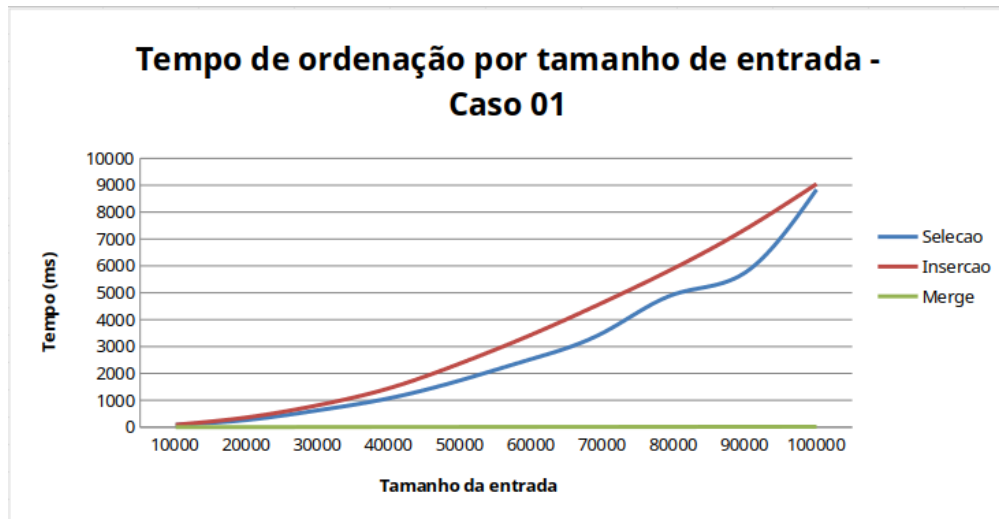


Figura 1 — Tempo de ordenação por tamanho de entrada (Caso 01 — Aleatório).

3.2 Caso 02 — Dados Já Ordenados

Quando os dados já estão ordenados, a função `insercao` (Código 3) mostra onde realmente ela se torna efetiva: como `a[idx]` nunca é menor que `a[j]` (linha 5), o bloco `else` com o `break` (linhas 11-13) é executado logo na primeira comparação de cada posição `i`, fazendo o laço interno rodar apenas uma vez por elemento — complexidade linear $O(n)$. Por isso os tempos são tão baixos (0,017 ms a 0,177 ms) que a curva sequer aparece de forma visível no gráfico, permanecendo colada ao eixo X. Já a função `selecao` (Código 2) não é muito efetiva aqui: o laço interno (linhas 4-8) sempre percorre todo o restante do vetor em busca do menor elemento, independentemente de ele já estar na posição correta.

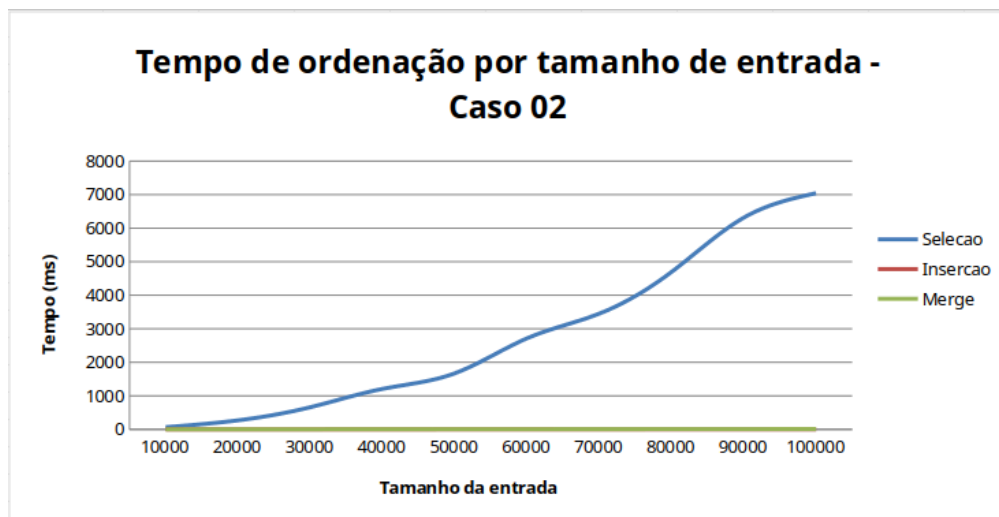


Figura 2 — Tempo de ordenação por tamanho de entrada (Caso 02 — Já ordenado).

3.3 Caso 03 — Dados em Ordem Inversa

Este é o pior caso para a função `insercao` (Código 3): como cada elemento é sempre menor que o anterior, a condição `a[idx]<a[j]` (linha 5) é sempre verdadeira, o `break` nunca é alcançado, e o laço interno percorre todas as posições de `j` até `-1` (linha 4). Por isso o Insertion se torna o algoritmo mais lento entre todos os cinco casos testados, superando inclusive a Seleção, que se mantém estável como nos demais cenários por não depender da ordem dos dados.

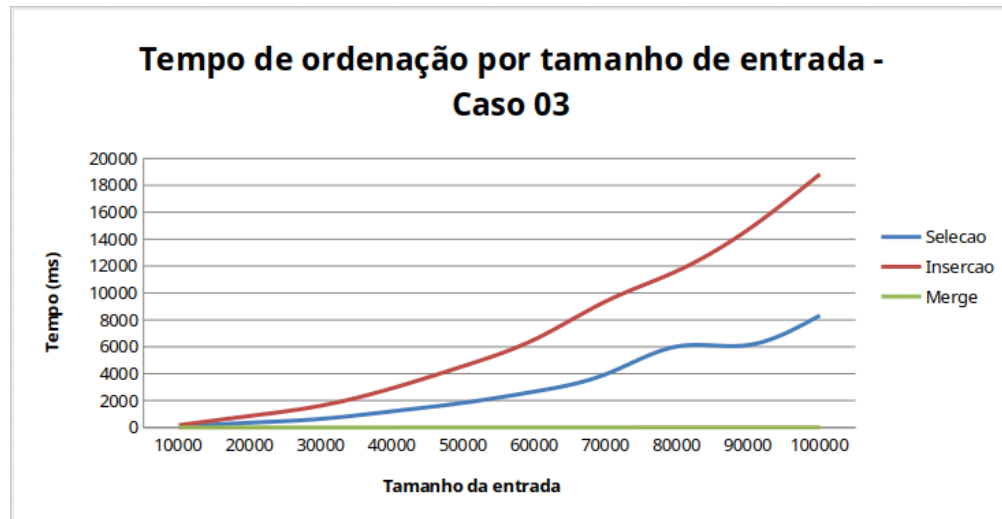


Figura 3 — Tempo de ordenação por tamanho de entrada (Caso 03 — Ordem inversa).

3.4 Caso 04 — Dados Quase Ordenados (Crescente)

Neste cenário, os dados estão organizados em blocos crescentes com pequenas desordens. A função `insercao` (Código 3) volta a se beneficiar fortemente da sua condição de parada antecipada (`break`, linha 12), já que a maioria dos elementos já está próxima de sua posição final, exigindo poucos deslocamentos. Isso resulta em tempos muito baixos (9,17 ms a 91,25 ms), embora não tão extremos quanto no Caso 02. A função `selecao` (Código 2), mais uma vez, não apresenta alteração relevante em relação aos demais casos, por sempre executar o laço interno completo.



Figura 4 — Tempo de ordenação por tamanho de entrada (Caso 04 — Quase ordenado crescente).

3.5 Caso 05 — Dados Quase Ordenados (Decrescente)

Este caso espelha o Caso 04, mas com tendência geral decrescente nos blocos de dados. Como a direção predominante dos dados é contrária à ordem final desejada, a condição $a[idx] < a[j]$ (linha 5 do Código 3) volta a ser verdadeira na maior parte das comparações, fazendo o break ser pouco acionado e aproximando o desempenho da função insercao do pior caso (Caso 03). Isso reforça que o fator determinante para o desempenho da Inserção não é apenas o grau de desordem local, mas principalmente a direção geral da sequência em relação à ordem final desejada.

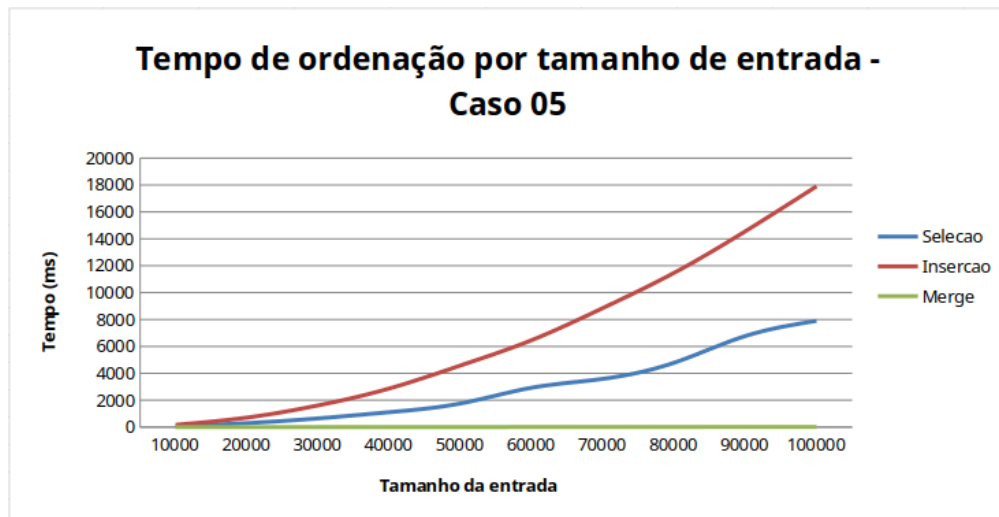


Figura 5 — Tempo de ordenação por tamanho de entrada (Caso 05 — Quase ordenado decrescente).

3.6 Tabela Resumo dos Resultados

A tabela a seguir apresenta os tempos de execução medidos para o menor (10.000) e o maior (100.000) tamanho de entrada em cada caso, além da razão de crescimento entre eles.

Caso	Método	10.000 (ms)	100.000 (ms)	Razão (100k/10k)
01 - Aleatório	Inserção	89,68	9.045,02	~101x
01 - Aleatório	Seleção	83,21	8.839,39	~106x
01 - Aleatório	Merge Sort	1,36	16,27	~12x
02 - Já ordenado	Inserção	0,017	0,177	~10x
02 - Já ordenado	Seleção	66,40	7.042,19	~106x
02 - Já ordenado	Merge Sort	0,83	9,95	~12x
03 - Ordem inversa	Inserção	179,03	18.832,71	~105x
03 - Ordem inversa	Seleção	70,13	8.329,32	~119x
03 - Ordem inversa	Merge Sort	0,81	10,25	~13x
04 - Quase ordenado	Inserção	9,17	91,25	~10x
04 - Quase ordenado	Seleção	67,02	7.224,62	~108x
04 - Quase ordenado	Merge Sort	1,21	13,96	~11,5x
05 - Quase inverso	Inserção	172,32	17.903,99	~104x
05 - Quase inverso	Seleção	70,04	7.899,83	~113x
05 - Quase inverso	Merge Sort	1,19	14,02	~11,8x

4. Resultados e Discussão

Os resultados obtidos confirmam integralmente o comportamento assintótico esperado para cada algoritmo, e podem ser explicados diretamente a partir da estrutura dos códigos apresentados na Seção 2.1:

Merge Sort (Código 4): apresentou o melhor desempenho em todos os cinco casos testados, coerente com sua complexidade $O(n \log n)$. Como `merge_sort_r` sempre divide o vetor pela metade dos índices (linha 34), sem examinar o valor dos elementos, a quantidade de chamadas recursivas e de comparações realizadas por merge independe da ordem dos dados de entrada, o que explica a estabilidade observada em todos os cenários.

Ordenação por Seleção (Código 2): manteve tempos de execução praticamente idênticos nos cinco casos, confirmando que o algoritmo é insensível à ordem dos dados de entrada. Isso ocorre porque o laço interno (linhas 4 a 8) sempre varre até o final do vetor em busca do menor elemento, resultando em complexidade $O(n^2)$ constante, sem melhor nem pior caso distinto.

Ordenação por Inserção (Código 3): foi o algoritmo com maior variação de desempenho entre os casos, evidenciando seu caráter adaptativo. No melhor caso (Caso 02, dados já ordenados), aproximou-se de complexidade linear $O(n)$, sendo até milhares de vezes mais rápida que nos demais cenários. No pior caso (Caso 03, ordem inversa), foi o algoritmo mais lento entre os três, superando até mesmo a Seleção, já que o break praticamente nunca é atingido. Nos casos intermediários de dados quase ordenados (Casos 04 e 05), seu desempenho variou proporcionalmente à direção da desordem: próximo ao melhor caso quando a tendência era crescente, e próximo ao pior caso quando a tendência era decrescente.

5. Conclusão

Fazer essa atividade toda me permitiu, antes de mais nada, com que eu conseguisse aprender melhor sobre os conceitos de cada tipo de ordenação, me ajudando na preparação das futuras avaliações e também a entender quando devo usar um algoritmo ou outro, já que cada um tem sua aplicação ideal para um caso específico. Sobre os 3 algoritmos, ficou claro que o Merge Sort é o mais eficiente e estável dos três (porém de difícil implementação). Já o Selection Sort se mostrou mais previsível e ineficiente para grandes volumes de dados, pois o laço não tem parada antecipada. O Insertion Sort, foi uma surpresa pra mim, pois dos três ele é o mais simples para implementar, e se mostrou bem adaptativo por conta do break, sendo uma boa escolha em casos onde o array está ordenado ou quase totalmente ordenado. Escolher o algoritmo ideal não depende somente da beleza de sua complexidade mas sim, onde e quando ele deve ser implementado.

6. Referências

VIDAL, J. M. B. Algoritmos: ordenação e divisão e conquista . Natal: Instituto Federal do Rio Grande do Norte (IFRN), 2026. Notas de aula (Slides disponibilizados no Google Sala de Aula).